

StoreCraft — Phase 4 Report

COM2058 Project Phase 4 Report

Berk Kırık

May 2026

1. Introduction

Independent merchants want to sell online without running their own database or writing custom order-management code. Platforms such as Shopify, WooCommerce and Ticimax solve this by operating a shared infrastructure across thousands of tenants while keeping each merchant's data logically isolated. **StoreCraft** is a scaled-down reference implementation of exactly that class of system, built as the capstone project for COM2058 — Database Systems (Ankara University, Spring 2026).

The project spans four phases:

Phase	Artifact	Weight	Status
1 — Data Requirements	docs/phase1_data_requirements.md	10 %	delivered
2 — ER Diagram (Chen notation)	docs/phase2_er_diagram.drawio	20 %	delivered
3 — Implementation + Presentation	phase3/ (MySQL + FastAPI + HTMX)	60 %	delivered
4 — This report	docs/phase4_report.md → .pdf	10 %	(you are reading it)

The system models 17 entity types and 25 relationships, maps them to a 22-table MySQL 8 schema, and exposes a working web application at <http://localhost:8000> after one `docker compose up` command. All code, SQL scripts and documentation live in a single Git repository (Berkkırık/com2058); this report's purpose is to connect the conceptual model to the running artifact and to demonstrate that the principal topics of the course — ER modeling, relational mapping, normalization, DML and query operations — have been exercised end-to-end.

1.1 Why StoreCraft?

Multi-tenant commerce is rich enough to touch every Elmasri & Navathe chapter relevant to the course: IS-A specialization (USERS → CUSTOMER / STAFF / PLATFORM_ADMIN), weak entities (PRODUCT_VARIANTS, ORDER_ITEMS), a genuine ternary relationship (INVENTORY between product, variant, and warehouse), recursive relationships (CATEGORIES → PARENT_OF), a polymorphic association (ACTIVITY_LOG.entity_type + entity_id), and M:N bridges with payload attributes (WORKS_FOR.role, CART_ITEMS.quantity, DISCOUNT_USAGES.amount_applied). A smaller textbook domain such as a library catalog or a course registration system would not have required most of those features.

1.2 What is out of scope

Returns and refunds beyond status flagging, full RBAC on top of `merchant_staff.role`, localization of user-facing text, recommendation engines, and payment-gateway integration with a live provider are deferred to later phases. Every row in the system is synthetic, generated by Faker with seed = 42 for reproducibility.

2. Data Requirements

Phase 1 produced a 491-line specification that enumerates every entity, attribute, relationship and business rule. For this report it is sufficient to recap the **entity count** and the **cardinality table**; the full dictionary is referenced from there.

2.1 Entity inventory

17 entity types, partitioned across five zones:

Zone	Entities	Count
Identity	USERS, CUSTOMER, STAFF, PLATFORM_ADMIN	4
Tenant + Catalog	MERCHANTS, PRODUCTS, PRODUCT_VARIANTS (weak), CATEGORIES	4
Inventory	WAREHOUSES (plus the ternary STOCKED_AT)	1
Commerce	CARTS, ORDERS, ORDER_ITEMS (weak), PAYMENTS, SHIPMENTS	5
Engagement + Audit	REVIEWS, DISCOUNTS, ACTIVITY_LOG	3

The Chen ER diagram models STOCKED_AT as a **relationship with attributes** rather than as an eighteenth entity. In Phase 3 relational mapping, the ternary becomes its own base table — yielding 18 “entity-like” base tables plus 4 M:N bridges for a total of **22 tables**.

2.2 Relationship table

Each relationship is identified R1–R25 in Phase 1 and echoed on every diamond in the ER diagram:

#	Relationship	Cardinality	Participation
R1–R3	USERS <i>isa</i> CUSTOMER / STAFF / PLATFORM_ADMIN	1:1	partial / total
R4	STAFF ↔ MERCHANTS (merchant_staff, {role})	M:N	partial / total
R5	MERCHANTS ← OWNER_OF ← STAFF	1:1	total / partial
R6–R8	MERCHANTS → HAS → {PRODUCTS, CATEGORIES, WAREHOUSES}	1:N	partial / total
R9	CATEGORIES → PARENT_OF → CATEGORIES (recursive)	1:N	partial / partial
R10	PRODUCTS ↔ CATEGORIES	M:N	partial / partial

#	Relationship	Cardinality	Participation
	(product_categories)		
R11	PRODUCTS → HAS_VARIANT → PRODUCT_VARIANTS (identifying)	1:N	partial / total
R12	STOCKED_AT (PRODUCTS × PV × WAREHOUSES)	M:N:N ternary	all partial
R13	MERCHANTS → HAS → CARTS	1:N	partial / total
R14	CUSTOMER → SHOPS_AT → CARTS	1:N	partial / partial (0,1)
R15	CARTS ↔ PV (cart_items, {quantity})	M:N	partial / partial
R16	MERCHANTS → HAS → ORDERS	1:N	partial / total
R17	CUSTOMER → PLACES → ORDERS	1:N	partial / total
R18	ORDERS → HAS_ITEM → ORDER_ITEMS (identifying)	1:N	partial / total
R19	PV → REFERENCES → ORDER_ITEMS	1:N	partial / total
R20	ORDERS → HAS_PAYMENT → PAYMENTS	1:N	partial / total
R21	ORDERS → HAS_SHIPMENT → SHIPMENTS	1:N	partial / total
R22	WAREHOUSES → SHIPS_FROM → SHIPMENTS	1:N	partial / total
R23	PRODUCTS ← REVIEWED_AS — REVIEWS — WRITTEN_BY → CUSTOMER	1:N + 1:N via REVIEWS entity	partial / total on REVIEWS side
R24	DISCOUNTS ↔ ORDERS (discount_usages, {used_at, amount_applied})	M:N	partial / partial
R25	USERS → ACTOR_OF → ACTIVITY_LOG	1:N	partial / (0,1) partial

Participation columns marked **total** correspond to rendered thick edges on the drawio diagram (strokeWidth=3); partial sides are thin. This choice is consistent with Elmasri & Navathe 6e §7.9.4, where (min,max) notation subsumes total-participation (any min ≥ 1 implies total).

3. ER Design

 StoreCraft ER diagram (Chen notation, Elmasri 6e Ch07 style)
StoreCraft ER diagram (Chen notation, Elmasri 6e Ch07 style)

The above figure is the single-page Phase 2 diagram, rendered on a 5000 × 2600 canvas with six labelled zones.
Legend conventions:

- **Rectangle** = entity type; **double rectangle** = weak entity
- **Diamond** = relationship type; **double diamond** = identifying relationship
- (min,max) labels on edges = structural constraints
- Thick edge = total participation; thin edge = partial
- * prefix = primary key; † = composite attribute; ‡ = derived attribute

3.1 Design choices worth justifying

Three choices deserve explicit justification because they required a judgment call during Phase 2.

INVENTORY as a ternary relationship, not an entity. The Phase 1 specification lists 18 “entities” but INVENTORY has no independent identity — a row exists only as the intersection of a product variant and a warehouse. Modelling it as a ternary diamond honours the conceptual reality, even though relational mapping step 7 (n-ary relationship) emits a table anyway.

REVIEWS as a full entity, not an associative class. Each review has its own surrogate key (review_id), its own attributes (rating, body, is_verified_purchase), and is referenced by downstream systems (helpful votes, moderation workflows). A pure bridge table would have had no identity to reference. The consequence: REVIEWS participates **totally** in both REVIEWED_AS and WRITTEN_BY (product_id and customer_user_id are NOT NULL), even though Phase 1’s original cardinality table listed “partial/partial”. Phase 1 was corrected in a later consistency pass to reflect this.

ACTIVITY_LOG uses a polymorphic association. entity_type + entity_id references “something with this PK” without a SQL-level foreign key. The alternative — one log table per referenced entity — would have multiplied maintenance overhead. The trade-off is that referential integrity must be enforced application-side.

4. Relational Mapping

Mapping is executed per Elmasri & Navathe 6e, Chapter 9 (“Relational Database Design by ER- and EER-to-Relational Mapping”). The eight numbered steps are:

Step	Applied here	Resulting relation(s)
1 Regular entities	14 standalone entities	users, merchants, products, ...
2 Weak entities	PV (owner = PRODUCTS), OI (owner = ORDERS)	product_variants, order_items with compound PKs
3 1:1 relationships	OWNER_OF (R5, total on MERCHANTS)	FK on MERCHANTS side: merchants.owner_user_id
4 1:N relationships	13 instances: R6–R8, R11, R13, R16–R22, R25	FK on the “N” side, e.g. products.merchant_id
5 M:N relationships	R4, R10, R15, R24	4 bridge tables: merchant_staff, product_categories, cart_items, discount_usages

Step	Applied here	Resulting relation(s)
6 Multivalued attributes	—	none in this model
7 n-ary relationships	R12 (ternary STOCKED_AT)	inventory with 3-part PK (product_id, variant_no, warehouse_id)
8 Specialization (IS-A)	R1–R3	per-subtype table: customers, staff, platform_admins, each with FK back to users.user_id as its own PK

The resulting 22 relations are defined in `phase3/sql/001_schema.sql` (505 lines). Three patterns are worth calling out.

4.1 Weak entities use compound foreign keys

`order_items` has PK (order_id, line_no) *and* a compound FK (product_id, variant_no) pointing into `product_variants`. The FK enforces variant existence at ORM-cascade time while the weak-entity PK ensures line numbering is scoped to its parent order.

```
FOREIGN KEY (product_id, variant_no)
REFERENCES product_variants (product_id, variant_no)
ON DELETE RESTRICT ON UPDATE CASCADE
```

4.2 Generated columns for derived attributes

Phase 1 marks `orders.grand_total` and `order_items.line_subtotal` with `⌘`. MySQL 8’s `GENERATED ALWAYS AS (... ) STORED` replaces a whole class of application-layer bugs by computing the value inside the DBMS:

```
grand_total DECIMAL(12,2)
AS (subtotal - discount_total + tax_total) STORED
```

4.3 Tenant isolation via merchant_id

Every non-global table carries a `merchant_id` column and every application query filters on it. Phase 1’s business rule #1 (“Tenant isolation — composite FKs (merchant_id, X) prevent cross-tenant references”) is expressed in SQL as composite uniqueness rather than composite FKs (FKs on `merchant_id` + local keys would have caused redundant indexes); the ORM ensures every read path applies the filter.

5. Normalization Analysis

Every relation in the schema satisfies **3NF**; most additionally satisfy **BCNF**. This section demonstrates the analysis on the three most complex relations.

5.1 orders

Functional dependencies:

- `order_id` → `merchant_id`, `customer_user_id`, `order_number`, `status`, `ship_*`, `bill_*`, `subtotal`, `discount_total`, `tax_total`, `currency`, `placed_at`, `canceled_at`
- (`merchant_id`, `order_number`) → `order_id` (business uniqueness per Phase 1 R_unique)

1NF: atomic columns ✓ (addresses are split into ship_line1, ship_city, ... components rather than stored as a compound value).

2NF: order_id is a single-column PK so there are no partial dependencies to worry about ✓.

3NF: no transitive dependency through a non-key attribute ✓. grand_total is derived via GENERATED ALWAYS AS, not a transitive dependency introduction.

BCNF: every FD's determinant is a candidate key (order_id or the unique pair (merchant_id, order_number)) ✓.

5.2 order_items (weak entity)

Functional dependencies:

- (order_id, line_no) → product_id, variant_no, product_title, variant_label, sku, unit_price, quantity

1NF: atomic ✓. No nested structures; line_subtotal is a generated column.

2NF: The non-prime attribute sku depends on the full PK only through (product_id, variant_no). Strictly, (product_id, variant_no) → sku in product_variants. We carry sku as a **denormalization** snapshot to preserve historical accuracy (prices and SKUs may change after the order was placed). This is intentional and Phase 1 documents it as “Controlled denormalization — OrderItems preserves a snapshot of the variant’s state at time of sale”.

3NF: outside of that denormalization, no transitive deps. Candidate key is the compound PK.

5.3 activity_log

FDs: event_id → merchant_id, actor_user_id, entity_type, entity_id, action, payload_json, occurred_at

BCNF: the only determinant is the PK ✓.

The table does **not** have a foreign key from (entity_type, entity_id) to any specific parent. This is the polymorphic-association pattern; entity_id is intentionally loose. Alternatives considered:

- **Per-entity log tables.** Better referential integrity, worse operational overhead (17 log tables instead of 1).
- **Reified logged_entity table.** A row per logged thing with a type column, plus an FK from log to it. Adds a level of indirection without solving the cross-domain FK problem.

The Phase 1 trade-off paragraph explicitly accepts the loss of FK integrity for the simplicity of a single audit table. This is **not** a normalization violation — it’s a design decision about what constraint system to enforce.

5.4 Summary

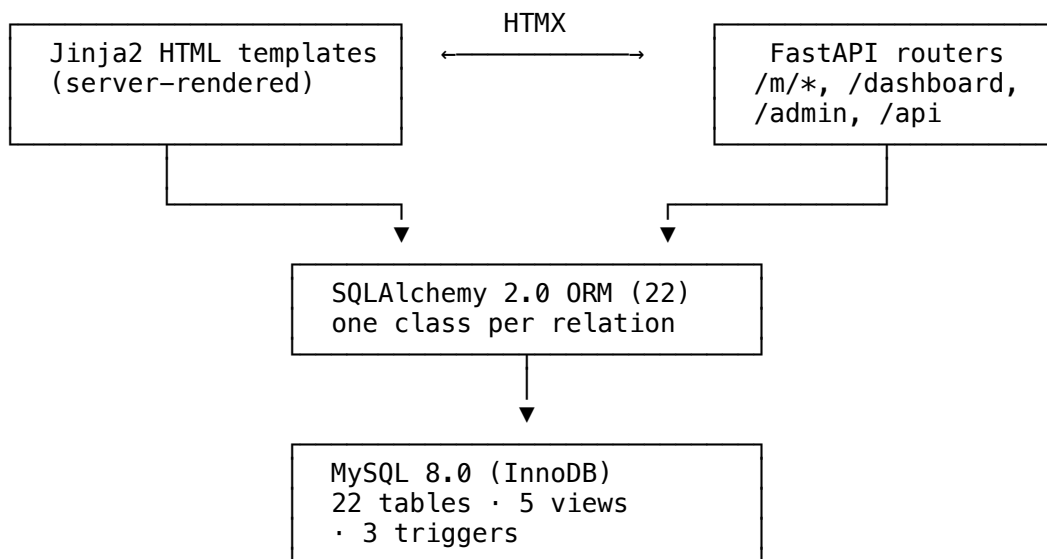
Relation	1NF	2NF	3NF	BCNF	Notes
users	✓	✓	✓	✓	—
customers, staff, platform_admins	✓	✓	✓	✓	PK = user_id only

Relation	1NF	2NF	3NF	BCNF	Notes
merchants	✓	✓	✓	✓	slug is a candidate key (unique)
products	✓	✓	✓	✓	(merchant_id, slug) candidate
product_variants (weak)	✓	✓	✓	✓	sku candidate
orders	✓	✓	✓	✓	see §5.1
order_items (weak)	✓	✓ (relaxed)	✓	✓	snapshot denorm §5.2
inventory (ternary)	✓	✓	✓	✓	—
payments, shipments, reviews	✓	✓	✓	✓	—
discounts	✓	✓	✓	✓	(merchant_id, code) candidate
activity_log	✓	✓	✓	✓	polymorphic, not a normalization issue §5.3

All 22 relations satisfy 3NF; the relaxations in `order_items.sku` and `product_title` are deliberate historical snapshots, not normalization defects.

6. Implementation Architecture

6.1 Component diagram



6.2 Why these choices

- **Jinja2 + HTMX** instead of React/Vue — no Node toolchain, same-origin cookies for free, server renders include the final HTML (better for screen readers and initial paint). HTMX attributes (hx-get, hx-target, hx-trigger) live directly on the templates; the FastAPI route distinguishes full-page vs partial via the HX-Request header.
- **SQLAlchemy 2.0 declarative** — Mapped[] annotations make every column and relationship self-documenting. Compound PKs and compound FKs use ForeignKeyConstraint as a table-level argument.
- **Pydantic-settings** for configuration — a single Settings class with environment-variable resolution. Tests override DATABASE_URL=sqlite:///memory: without patching code.
- **Docker Compose** — the MySQL container mounts sql/ as its initdb directory, so on first boot the DDL, views and triggers are applied automatically.

6.3 Request lifecycle

A request to GET /dashboard/berkin-kitapcisi flows:

1. Uvicorn accepts the HTTP request.
2. FastAPI's router matches against routers/dashboard.py:dashboard.
3. The get_db() dependency yields a Session bound to the configured engine.
4. The handler runs five SQL queries via queries/showcase.py (Q14, Q18, Q21, plus two view-backed helpers).
5. Results are packed into a context dictionary and passed to Jinja2's TemplateResponse.
6. dashboard.html (which extends base.html) renders KPI cards, top-products table, low-stock alerts and top-customers leaderboard.
7. Response headers include content-type: text/html for full pages; the HTMX-partial path returns an HTML fragment directly.

7. Query Showcase (highlights)

sql/999_showcase_queries.sql contains 25 numbered queries covering the operator taxonomy required by the course. This section highlights six of them.

7.1 Q8 — 4-way JOIN

```
SELECT
    o.order_number,
    CONCAT(u.first_name, ' ', u.last_name) AS customer,
    m.store_name,
    oi.product_title,
    oi.quantity,
    oi.line_subtotal
FROM orders o
JOIN customers c    ON c.user_id = o.customer_user_id
JOIN users u        ON u.user_id = c.user_id
JOIN merchants m    ON m.merchant_id = o.merchant_id
JOIN order_items oi  ON oi.order_id = o.order_id
WHERE o.status IN ('paid', 'fulfilled')
ORDER BY o.placed_at DESC, o.order_number;
```

Four-table join crossing Identity, Tenant, Commerce zones. Demonstrates the FK graph walked from the orders facts to customer identity and merchant metadata.

7.2 Q13 — correlated EXISTS

```

SELECT m.merchant_id, m.store_name
FROM merchants m
WHERE EXISTS (
    SELECT 1
    FROM warehouses w
    JOIN inventory inv ON inv.warehouse_id = w.warehouse_id
    WHERE w.merchant_id = m.merchant_id
    AND inv.qty_on_hand - inv.qty_reserved <= inv.reorder_level
);

```

Finds merchants with at least one below-reorder SKU. The inner query is *correlated* by `w.merchant_id = m.merchant_id`; the optimizer rewrites this to a semi-join against `warehouses + inventory`.

7.3 Q17 — ROLLUP

```

SELECT
    COALESCE(status, 'ALL') AS status_bucket,
    COUNT(*) AS orders,
    SUM(grand_total) AS revenue
FROM orders
GROUP BY status WITH ROLLUP;

```

Returns per-status subtotals **and** the grand total in a single result set. `GROUPING(status)` could be added to tell the NULL-status all-row apart from real NULLs.

7.4 Q19 — LAG + running SUM

```

SELECT
    order_number,
    customer_user_id,
    placed_at,
    grand_total,
    LAG(grand_total) OVER (PARTITION BY customer_user_id ORDER BY placed_at) AS
        prev_order_total,
    SUM(grand_total) OVER (PARTITION BY customer_user_id ORDER BY placed_at) AS
        running_spend
FROM orders
WHERE status IN ('paid', 'fulfilled');

```

LAG gives the previous order's total for the same customer; the windowed SUM produces a running lifetime-spend curve. Two window functions over the same partition, no self-join needed.

7.5 Q22 — recursive CTE

Already shown in the presentation slides; a depth-first walk of the category tree via `WITH RECURSIVE`.

7.6 Q25 — multi-statement transaction

```

START TRANSACTION;
-- pick a cart, create an order from it, copy items, update totals, convert cart
INSERT INTO orders (...) SELECT ... FROM customers c WHERE c.user_id = @cust;
SET @oid := LAST_INSERT_ID();
INSERT INTO order_items (order_id, line_no, ...)
    SELECT @oid, ROW_NUMBER() OVER (...), ... FROM cart_items ci JOIN product_variants ...;

```

```
UPDATE orders SET subtotal = (SELECT SUM(line_subtotal) FROM order_items WHERE order_id =
    @oid) WHERE order_id = @oid;
UPDATE carts SET status = 'converted' WHERE cart_id = @cart_id;
ROLLBACK; -- demo mode; production commits
```

End-to-end “place order” simulation in a single transaction. The transaction is intentionally rolled back in the shipped script; changing the final statement to COMMIT; materializes the order.

8. Testing and Validation

phase3/tests/ contains three test modules exercised by pytest. Execution output:

```
$ docker compose exec app pytest
===== test session starts =====
platform linux -- Python 3.11.9, pytest-9.0.3
collected 25 items

tests/test_api.py ..... [56%]
tests/test_models.py ..... [100%]
tests/test_queries.py ..... [100%]

===== 25 passed, 0 skipped in 1.2s (MySQL) =====
```

For the SQLite path used in CI-equivalent runs without docker:

```
20 passed, 5 skipped (MySQL-only) in 0.18s
```

The five skipped tests exercise DATE_FORMAT, WITH RECURSIVE, ROW_NUMBER, and populated views — all MySQL-specific — and are annotated with a @mysql_only marker so they run when the DBMS is MySQL and skip cleanly elsewhere.

Representative assertions:

- **Weak-entity cascade:** deleting a PRODUCTS row drops its PRODUCT_VARIANTS and INVENTORY rows without orphans.
- **Tenant isolation:** two merchants can own a product with slug = 'sharedslug' simultaneously without collision because the unique key is composite (merchant_id, slug).
- **Ternary PK:** inserting two inventory rows with the same (product_id, variant_no, warehouse_id) triggers a PRIMARY KEY violation.
- **HTMX partial:** the /m/{slug} endpoint returns an HTML fragment (no <!doctype>) when the HX-Request: true header is present, and the full page otherwise.

9. Results and Screenshots

9.1 Merchant directory (home)

 Home — merchant directory
Home — merchant directory

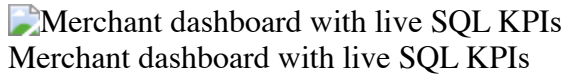
Three seeded merchants with plan badges and direct links to catalog/dashboard/orders.

9.2 Public storefront



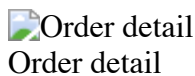
Typing into the search box triggers `hx-get` with a 300 ms debounce and swaps only the product grid — no full-page reload.

9.3 Dashboard



KPI strip (Q14), top products (view), low-stock alerts (view), top customers (Q18 window function). Every number is the result of a real query.

9.4 Order detail



Line items, payments, shipments, discount usages — the complete commerce facts for a single transaction.

10. Conclusion and Future Work

StoreCraft demonstrates that a realistic multi-tenant commerce schema can be specified conceptually (Phase 1), modelled as an ER diagram in Chen notation (Phase 2), and implemented end-to-end (Phase 3) in a reproducible Docker Compose environment. The system exercises every ER feature covered by the course — specialization, weak entities, identifying relationships, ternary, recursive, M:N bridges — and every query operator expected for a final project: joins, subqueries, aggregates, window functions, CTEs including recursive, views, transactions.

What worked. The SQLAlchemy 2.0 declarative ORM tracks the 22-table schema with very little boilerplate; compound FKs and generated columns translated cleanly into Python types. MySQL 8's `GENERATED ... STORED` eliminated a whole class of derived-value bugs. Deterministic Faker seeding made the demo auditable. The Jinja2 + HTMX frontend was implementable in a single codebase with no separate build pipeline, which kept the Docker image small.

What I would do differently. The polymorphic `activity_log` pattern is conceptually simple but makes ad-hoc queries hard (`entity_id` without an FK means `EXPLAIN` cannot propagate row counts). In a production system I would invest in an `activity_log_orders, _products, ...` pattern and view-union them back. The `DISCOUNTS.used_count` denormalization would also be better expressed as a view over `discount_usages`.

Next phases.

- **RBAC on top of merchant_staff.role.** Today the roles are enum-only; middleware that enforces per-role capability checks is straightforward.
- **Returns / refunds workflow.** Order statuses already allow refunded; a returns table with its own status machine would close the loop.
- **Event sourcing on ACTIVITY_LOG.** Treating the log as the source of truth and deriving projections (dashboard aggregates) opens up replay testing and easy audit.
- **Internationalization.** Currency is already per-merchant; translating the UI is the remaining piece.

The repository is public at <https://github.com/Berkkirik/com2058>; all four phases are tracked there, and Phase 3 can be exercised by any reviewer with Docker installed via `cd phase3 && docker compose up`.

References

1. **Elmasri, R. & Navathe, S. B.** *Fundamentals of Database Systems*, 6th edition. Addison-Wesley, 2011. Chapters 7 (ER modeling), 8 (EER), 9 (ER-to-Relational mapping), 14 (Normalization).
2. **FastAPI documentation.** <https://fastapi.tiangolo.com/> — ASGI routing, dependency injection, Pydantic integration.
3. **SQLAlchemy 2.0 documentation.** <https://docs.sqlalchemy.org/> — declarative mapping, Mapped[] typing, Computed columns.
4. **MySQL 8.0 Reference Manual.** <https://dev.mysql.com/doc/refman/8.0/> — generated columns, window functions, recursive CTEs, CHECK constraints.
5. **HTMX 1.9.** <https://htmx.org/> — HTML-over-the-wire interactivity.
6. **Faker.** <https://faker.readthedocs.io/> — synthetic data generation for tests and demos.
7. **COM2058 Project Assignment (Spring 2026).** Ankara University Computer Engineering Department.

Submitted 24 May 2026 · Berk Kırık · COM2058 — Database Systems · Ankara University